

Task –1 Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

Code:

```
rohita@LaxmiDurga: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <stdlib.h>  
#include <unistd.h>  
  
int global_var = 10;  
static int static_var = 20;  
  
void code_function()  
{  
    printf("This is a code function.\n");  
}  
  
int main()  
{  
    int stack_var = 30;  
    static int local_static_var = 40;  
  
    int *heap_var = (int *)malloc(sizeof(int));  
    *heap_var = 50;  
  
    printf("Process ID: %d\n\n", getpid());  
  
    printf("Code address      : %p\n", (void *)code_function);  
    printf("Global variable   : %p\n", (void *)&global_var);  
    printf("Static variable    : %p\n", (void *)&static_var);  
    printf("Local static variable : %p\n", (void *)&local_static_var);  
    printf("Heap variable      : %p\n", (void *)heap_var);  
    printf("Stack variable     : %p\n", (void *)&stack_var);  
  
    printf("\nProcess is running...\n");  
    printf("Press Ctrl+C to stop.\n");  
  
    while (1)  
    {  
        sleep(2);  
    }  
  
    free(heap_var);  
  
    return 0;  
}
```

Output:

```
/home/rohita/.hushlogin file.  
rohita@LaxmiDurga:~$ nano p7.c  
rohita@LaxmiDurga:~$ gcc p7.c  
rohita@LaxmiDurga:~$ ./a.out  
Process ID: 870  
  
Code address      : 0x64cf978db1c9  
Global variable   : 0x64cf978de010  
Static variable   : 0x64cf978de014  
Local static variable : 0x64cf978de018  
Heap variable     : 0x64cfd59a6010  
Stack variable    : 0x7ffde6d0a9fc  
  
Process is running...  
Press Ctrl+C to stop.
```

observation:

Task 2: Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

Code:

```

rohita@LaxmiDurga: ~
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_var = 10;
static int static_var = 20;

void display()
{
    printf("Inside code segment\n");
}

int main()
{
    int stack_var = 30;
    static int local_static = 40;

    int *heap_var = malloc(sizeof(int));
    *heap_var = 50;

    printf("Process ID: %d\n", getpid());

    printf("Code address      : %p\n", (void *)display);
    printf("Global variable   : %p\n", (void *)&global_var);
    printf("Static variable    : %p\n", (void *)&static_var);
    printf("Local static variable : %p\n", (void *)&local_static);
    printf("Heap variable      : %p\n", (void *)heap_var);
    printf("Stack variable     : %p\n", (void *)&stack_var);

    printf("\nProcess is running...\n");

    while (1)
    {
        sleep(2);
    }

    free(heap_var);

    return 0;
}

```

Output:

```

rohita@LaxmiDurga:~$ nano p71.c
rohita@LaxmiDurga:~$ gcc p71.c
rohita@LaxmiDurga:~$ ./a.out
Process ID: 911
Code address      : 0x5ee247cb61c9
Global variable   : 0x5ee247cb9010
Static variable   : 0x5ee247cb9014
Local static variable : 0x5ee247cb9018
Heap variable     : 0x5ee2643a9010
Stack variable    : 0x7fffeb8d6d7c

Process is running...

```

Observation: The program successfully displays the addresses of code, global, static, heap, and stack variables, illustrating the different memory segments in the Linux process address space.

When you run this program, it prints memory addresses that illustrate how the operating system lays out process memory in Virtual Address Space (VAS).